

Week 3 - Day 13a: Merge Sort

In this lab, we will focus on sorting algorithms and their implementation. Sorting is a fundamental concept in computer science that helps in organizing data in a particular order. Today we will focus on merge sort, a powerful divide-and-conquer algorithm that uses the concept of merging smaller sorted lists into larger sorted lists to sort any list.

Let's get started with the exercises!

Exercise 1: Incremental Sorting [] []

In this exercise, we will step-by-step implement sorting functions for lists of different lengths and combine them to sort larger lists.

1.1 Write a function `sort_two` that sorts a list of numbers with a length of 2.

```
# TODO: Your code here
def sort_two(lst):
    pass

# Use this cell to test your code
assert sort_two([37, 9]) == [9, 37]
assert sort_two([1, 29]) == [1, 29]

print("All tests passed! [ ]")
```

1.2 Nice! Now we can sort two elements. Let's move on to sorting four elements. In your function `sort_four` you should make two calls to `sort_two` and then combine the results.

```
# TODO: Your code here
def sort_four(L):
    left = sort_two(L[0:2])
    right = sort_two(L[2:4])
    pass

# Use this cell to test your code
assert sort_four([37, 9, 15, 2]) == [2, 9, 15, 37]
assert sort_four([1, 29, 10, 5]) == [1, 5, 10, 29]
assert sort_four([100, 50, 75, 25]) == [25, 50, 75, 100]
assert sort_four([4, 3, 2, 1]) == [1, 2, 3, 4]
assert sort_four([1, 1, 2, 2]) == [1, 1, 2, 2]

print("All tests passed! [ ]")
```

1.3 Great, we can keep this going. Now, let's sort 8 elements. In your function `sort_eight` you should make two calls to `sort_four` and then combine the results.

```

# TODO: Your code here
def sort_eight(L):
    left = sort_four(L[0:4])
    right = sort_four(L[4:8])

    pass

# Use this cell to test your code
assert sort_eight([8, 3, 2, 5, 1, 7, 6, 4]) == [1, 2, 3, 4, 5, 6, 7, 8]
assert sort_eight([100, 50, 75, 25, 37, 9, 15, 2]) == [2, 9, 15, 25, 37, 50, 75, 100]
assert sort_eight([1, 1, 2, 2, 3, 3, 4, 4]) == [1, 1, 2, 2, 3, 3, 4, 4]
assert sort_eight([10, 9, 8, 7, 6, 5, 4, 3]) == [3, 4, 5, 6, 7, 8, 9, 10]
assert sort_eight([5, 5, 5, 5, 1, 1, 1, 1]) == [1, 1, 1, 1, 5, 5, 5, 5]

print("All tests passed! ☐")

```

1.4 Did you write any new code in `sort_eight` compared to `sort_four`? Does the logic change as you increase the list size?

Exercise 2: Merging ☐☐☐

As you have seen in the previous exercise, we can combine two sorted lists into one sorted list and use this logic to sort larger and larger lists. This process is called **merging**. First, extract your merging logic into a function, so that we can use it later in **merge sort**, the sorting algorithm that depends on merging.

You can use your code from `sort_four` and `sort_eight` as a starting point. But, be careful - merging two arbitrary lists is different from merging two lists of length 2 or 4. Think about what could go wrong in the case of arbitrary lists.

```

# TODO: Your code here
def merge(lst1, lst2):
    pass

# Use this cell to test your code
assert merge([1, 3, 5], [2, 4, 6]) == [1, 2, 3, 4, 5, 6]
assert merge([1, 2, 3], [4, 5, 6]) == [1, 2, 3, 4, 5, 6]
assert merge([], [1, 2, 3]) == [1, 2, 3]
assert merge([1, 2, 3], []) == [1, 2, 3]
assert merge([1, 3, 5, 7], [2, 4, 6]) == [1, 2, 3, 4, 5, 6, 7]
assert merge([1, 2, 2, 3], [2, 3, 4, 5]) == [1, 2, 2, 2, 3, 3, 4, 5]
assert merge([-3, 0, 5], [-2, 1, 4]) == [-3, -2, 0, 1, 4, 5]

print("All tests passed! ☐")

```

Exercise 3: Merge Sort [] [] []

Beautiful, equipped with our `merge` function and with the realisation that if we can merge two sorted lists into one sorted list, we can sort any list we want. This is the main idea behind merge sort.

Write a function `mergesort` that sorts a list `lst` using the `merge` function. Think about the base case and the recursive case.

```
# TODO: Your code here
def mergesort(lst):
    pass

# Testing the mergesort function
assert mergesort([]) == []
assert mergesort([1]) == [1]
assert mergesort([23, 10, 5, 1, 4]) == [1, 4, 5, 10, 23]
assert mergesort([4, 3, 2, 1]) == [1, 2, 3, 4]
assert mergesort([2, 2, 2, 2]) == [2, 2, 2, 2]
assert mergesort([-5, 0, 10, -3, 8]) == [-5, -3, 0, 8, 10]

print("All tests passed! [ ]")
```

Congratulations []